Manual Tecnico

Simulador de EXT2 / EXT3

Guia para el desarrollador (Frontend y Backend)

Proyecto 2
Manejo e Implementación de Archivos
Gisela Mishell Monroy Ovalle
202404409

ÍNDICE

Introducción..... 3
Especificación técnica.....4
Requisitos de hardware.....4
Requisitos de Software..... 4
Arquitectura general..... 4
Flujo del sistema..... 5
Flujo de una petición..... 6
Despliegue en AWS..... 7
Amazon Elastic Compute Cloud (EC2)..... 7
Pasos para el despliegue en AWS:..... 7
Consideraciones.....8
Compilación y ejecución local..... 8
Persistencia, límites y recomendaciones..... 9
Archivos importantes.....9
Backend..... 10
Comandos utilizados:..... 11
estado.h (permanece en RAM)..... 12
structs.h (estructuras binarias)..... 12
Analizador (léxico y sintáctico)..... 12
Tokenización.....12
Despacho..... 12
Archivo .smia.....12
Simulación EXT2 / EXT3: cómo se representa en el archivo binario .mia..... 13
MKDISK: creación física del disco..... 13
Estructuras de datos..... 13
MBR (Master Boot Record)..... 13
EBR (Extended Boot Record)..... 14
Superbloque..... 14
Bitmaps..... 14
Inodos.....14
Bloques..... 14
Cálculo utilizado por MKFS..... 15
Campos de ubicación del superbloque..... 15
Inodos, bloques y bitmaps.....15
Inodo..... 15
Bloque de carpeta..... 15
Bloque de archivo.....16
Asignación.....16
Usuarios, grupos, archivos y permisos..... 16
MKFS crea users.txt.....16
LOGIN..... 16
MKGRP / RMGRP / MKUSR / RMUSR / CHGRP.....16

MKDIR y MKFILE..... 17
Permisos..... 17
Reportes REP..... 17
Interfaz gráfica.....18
App.jsx.....18
api.js (puente entre React y C++)..... 18
main.jsx, index.htm, App.cpp y package.json..... 19
Archivo .smia.....19
API REST y comunicación Frontend/Backend.....19
POST / api / procesar paso a paso..... 19
http_server.cpp.....19
json_util.cpp..... 20
MAKEFILE.....20
Variables y módulos comunes.....20
Plan de recomendaciones.....21
Acciones recomendadas para su implementación.....22
Plan de mantenimiento.....22
Mantenimiento preventivo..... 23
Mantenimiento correctivo..... 24
Mantenimiento evolutivo.....25
Procedimiento de mantenimiento.....25
Indicadores para evaluar el mantenimiento..... 26

Introducción

Este es un simulador de un sistema de archivos de tipo EXT2 y con soporte para EXT3, se simula la creación de discos, de particiones, el montaje de estas particiones, inicio de sesión al sistema, generación de reportes, etc. Toda la información es manejada por medio de la creación de archivos binarios con extensión .mia, de tamaño fijo, aquí se simula la creación de discos, el montado, formateado, eliminado, etc. Este proyecto se realizó con implementación en la nube utilizando AWS.

Objetivos:

Brindar una idea clara de cómo fue creado el programa, para que cualquier otra persona pueda interpretar de una buena manera el código creado, incluso es viable ver este manual para cuando se le desea hacer una actualización el programa, se sabe dónde se debe de hacer, debido a que en este documento se explica de forma clara y ordenada las funciones del código, también busca Explicar qué hace cada capa, qué archivo es responsable de cada tarea, cómo viaja un comando desde la interfaz hasta C++, cómo funciona el Makefile y cómo una partición EXT2 y EXT3 se representa físicamente dentro de un .mia.

Dirigido a:

Está dirigido a todas aquellas personas que deseen modificar o actualizar el programa, también para quienes deseen un programa como este, para que sepan cómo deben leer el código y como es su funcionamiento, incluso está dirigido a el programador en sí, ya que de esta forma puede saber de una manera más clara y ordenada que es y dónde está el código que desea implementar o actualizar.

Especificación técnica

Requisitos de hardware

- Disco duro capaz de correr el programa, a partir de 250 GB
- Procesador Core i5 o superior
- 4GB de memoria RAM como mínimo.

Requisitos de Software

- Debe tener instalado algún editor de código fuente, para usar el programa directamente, Computadora de escritorio o portátil, se recomienda utilizar Visual Studio Code.
- Instalar previamente las dependencias necesarias, específicamente:
    - C++ (lenguaje de programación con el que se desarrolló todo)
    - npm para correr el servidor
    - make (para la api)
    - Usuario en AWS para utilizar EC2 Y S3

Arquitectura general

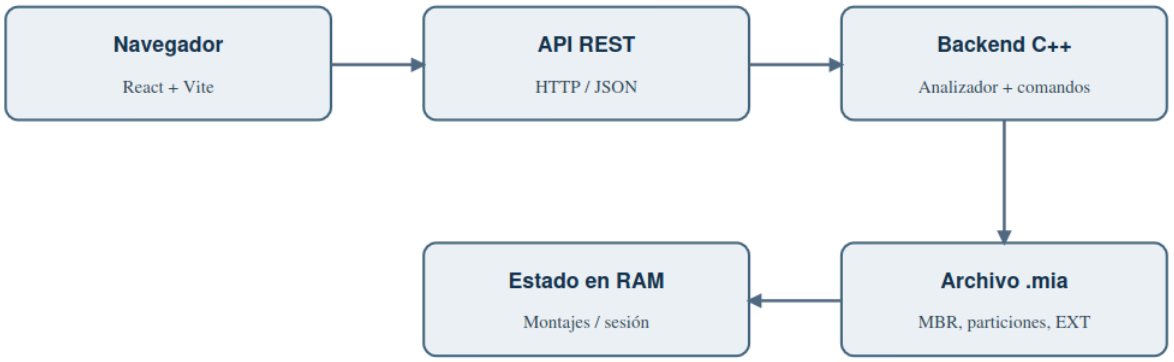
El proyecto está dividido en 5 capas: interfaz web, API/servidor HTTP, núcleo del backend, almacenamiento persistente y la nube. El frontend envía una petición HTTP a la API. La API entrega el texto al analizador, el analizador válida y despacha el comando, y el comando modifica el estado correspondiente, incluyendo el archivo .mia cuando la operación es persistente.

Capa	Responsabilidades	Archivos principales
Frontend	Captura comandos y archivos .smia y muestra los resultados	Frontend/src/App.jsx, api.js, main.jsx, App.css
API/HTTP	Recibe HTTP, extrae JSON, llama al analizador y devuelve un JSON	api_server_main.cpp, http_server.cpp, json_util.cpp
Analizador	Tokeniza, válida léxicamente y sintácticamente, despacha	analizador.cpp, validaciones.cpp
Comandos	Implementan la semántica de MKDISK, FDISK, MOUNT y MKFS	mkdisk.cpp, fdisk.cpp, mount.cpp, mkfs.cpp…
EXT2 /	Lee/escribe superbloque, bitmaps,	ext2_utils.cpp,

Persistencia	inodos y bloques en .mia	disk_utils.cpp, structs.h
Build	Compila consola y API con los mismos módulos comunes	Makefile
AWS	Sube el proyecto a la nube	por medio de EC2 Y S3

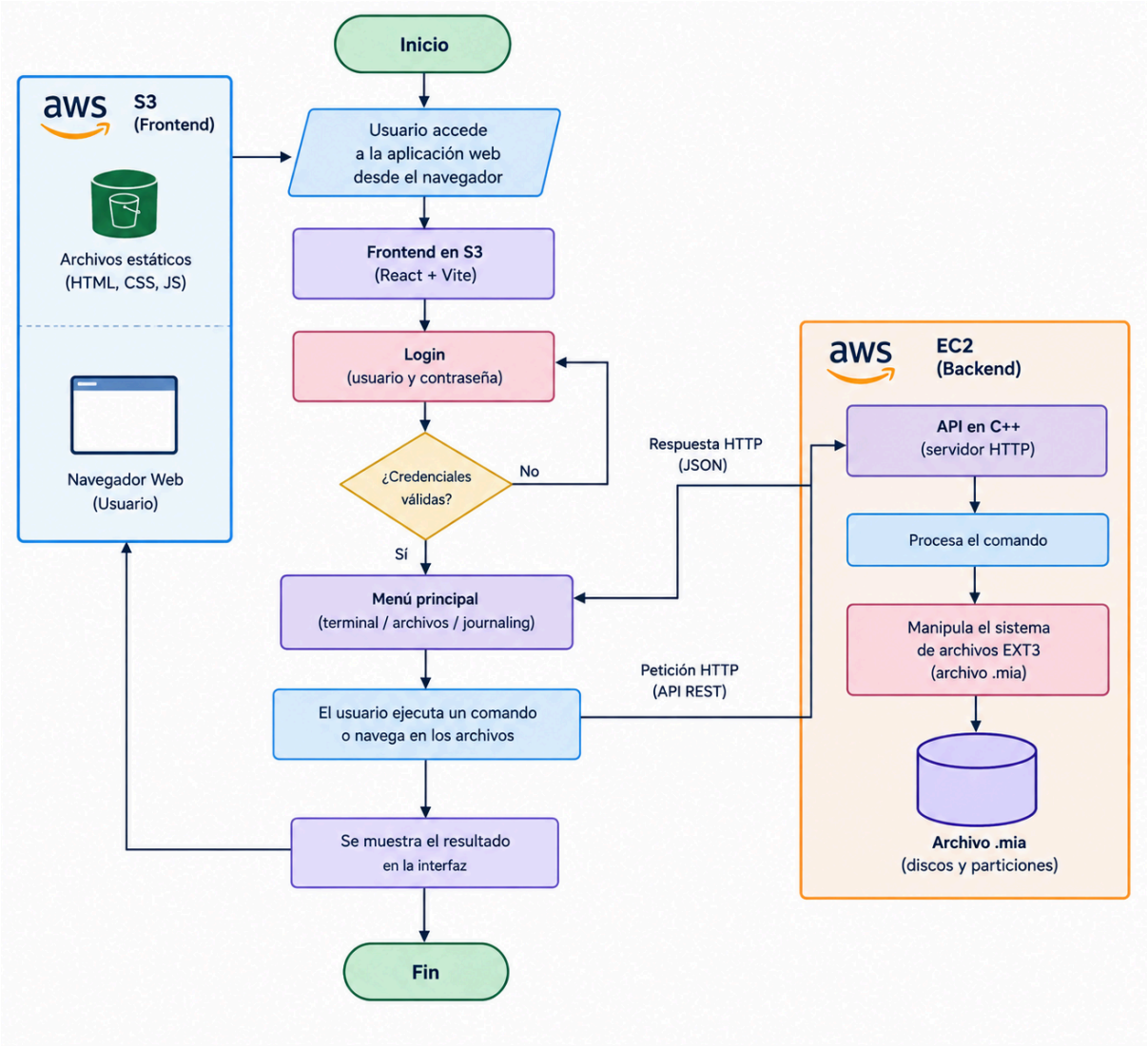
Flujo del sistema

1. **React**: app.jsx mantiene el texto y llama a api.js
2. **HTTP**: fetch haec POST/api/procesar a localhost: 8080
3. **API C++**: api_server_main.cpp recibe JSON y llama a procesar_archivo_comandos().
4. **Analizador**: por medio de lexical_analysis valida las comillas, valida el lexico, luego la sintaxis y por último despacha, con dispatcher.
5. **Comando**: Ejecuta semántica y si corresponde lee/escribe en el .mia
6. **Respuesta**: el resultado textual vuelve como JSON.
7. **React**: App.jsx coloca el reporte en el textarea de Resultados.
8. **AWS**: se carga el frontend en S3 y se hacen peticiones al backend por medio de EC2



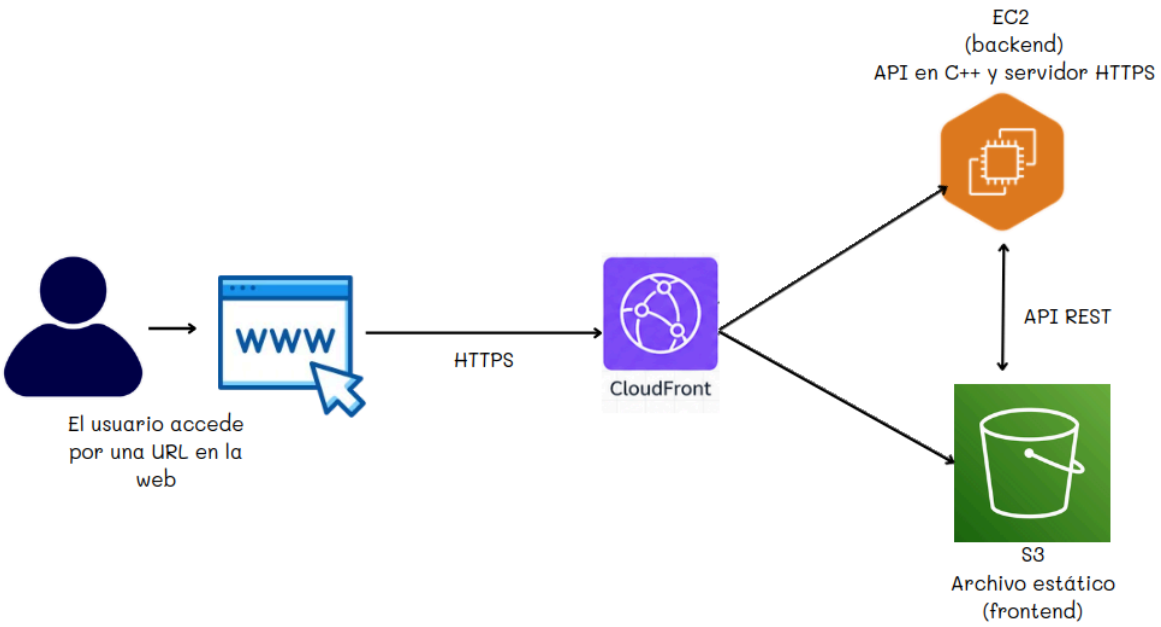
Flujo de una petición

1. El usuario ejecuta un comando.
2. El frontend envía una petición, por ejemplo POST /api/procesar, con JSON.
3. La API entrega el texto al analizador.
4. El analizador valida y ejecuta el comando.
5. El módulo consulta o modifica el .mia y/o el estado en RAM.
6. La API devuelve el resultado y el frontend lo muestra.



Despliegue en AWS

El despliegue debe mantener accesible el frontend desde el navegador y permitir que este se comunique con el backend por HTTP. El siguiente esquema es una arquitectura de referencia; ajústese a los servicios realmente usados por el equipo.



Amazon Elastic Compute Cloud (EC2)

Amazon Elastic Compute Cloud (Amazon EC2) ofrece la plataforma de computación más amplia y profunda, con más de 600 tipos de instancias y una selección de los últimos procesadores, almacenamiento, redes, sistemas operativos y modelos de compra de modo que se satisfagan al máximo las necesidades de la carga de trabajo. Es para crear, administrar y supervisar servidores virtuales en la nube.

Pasos para el despliegue en AWS:

1. Crear tu cuenta en AWS

2. Recomendable: crear tu perfil IAM, y autorizar únicamente los permisos necesarios:
    a. EC2 full access
    b. SE full access
    c. VPC full access
    d. Cloud full total access

3. Siempre antes de crear cualquier servicio es recomendable crear antes una VPC, para evitar problemas con inconvenientes de seguridad o privacidad,

para esto busco VPC -> crear una nueva -> configurarla -> 1 subnet pública , 0 subnet privada, no vpc endpoint ni gateway -> crear

4. Buscar EC2 -> instancias -> lanzar instancia -> configurar (ubuntu, t3 micro, key pair, configuración de red, seleccionar la propia VPC, habilitar IP, crear grupos de seguridad, agregar una extra security con puerto 8080) -> lanzar instancia -> esperar a que todos los campos se verifiquen -> cliente SSH -> copiamos la URL

5. En la consola del proyecto -> restringir los permisos de la llave -> pegar la llave de ssh hacia la ec2, cambia la terminal a la instancia de AWS -> actualizar todo usando sudo apt update

6. Buscar S3 -> colocar namespace-> no bloquear acceso público, no versionamiento y encriptación -> crear bucket -> propiedades (colocarlo como página estática, colocar html en el index, para el error, que regrese al index) -> copiar el link que carga la aplicación

7. En .env pegar la dirección pública de la instancia de EC2 -> Crear distribución en la terminal con npm build -> instalar las instancias necesarias -> correr el backend y listo, ya está levantado el servidor.

Consideraciones

VITE_API_URL debe apuntar a la URL alcanzable del backend; no a localhost cuando el usuario navega desde otra computadora.
Las variables VITE_* son visibles en el código del navegador y no deben contener secretos. En AWS, abra solo los puertos
necesarios, limite CORS al origen del frontend y prefiera HTTPS. Guarde .mia en almacenamiento persistente si debe sobrevivir a
reinicios o reemplazos de instancia. Las rutas locales deben existir en el servidor.

Compilación y ejecución local

Backend (Linux), desde Backend:

```
make clean\nmake\nmake test
```

Frontend, desde Frontend:

```
npm install\nnpm run dev
```

La variable VITE_API_URL debe coincidir con la dirección y puerto donde escucha la API. En producción no debe apuntar a localhost si el navegador se ejecuta en otra computadora.

Persistencia, límites y recomendaciones

- El archivo .mia contiene las estructuras persistentes; conserve copias de seguridad.
- Montajes y sesión pueden perderse al reiniciar el backend; vuelva a ejecutar MOUNT y LOGIN.
- Use comandos destructivos únicamente en discos de prueba.
- Restrinja CORS, puertos y acceso a comandos antes de publicar la API en Internet.
- Las garantías del journaling dependen de las funciones de recuperación realmente implementadas.
- Verifique los parámetros exactos en el analizador/validador antes de la entrega.

Archivos importantes

1. Frontend: Frontend/src/App.jsx, api.js, App.css.
2. API: Backend/api_server_main.cpp, http_server.cpp, json_util.cpp.
3. Analizador: Backend/analizador.cpp, validaciones.cpp.
4. Estructuras/persistencia: Backend/structs.h, disk_utils.cpp, ext2_utils.cpp, mkfs.cpp.
5. Journaling: Backend/journaling.cpp, journal_command.cpp.
6. Reportes: Backend/reports.cpp.
7. Compilación/pruebas: Backend/Makefile y Backend/tests/

Backend

El Backend está organizado por responsabilidad. Los archivos .h declaran estructuras y funciones; los .cpp implementan la lógica. La parte central puede entenderse en seis grupos.

Grupo	Responsabilidad	Archivos principales
Entrada	Modo consola	main.cpp
Analizador	Tokenización, procesamiento de líneas y despacho	analizador.cpp y .h
Validación	validación léxica, sintáctica y comillas	validaciones.cpp y .h
Estado / estructuras	estado compartido en RAM y estructuras binarias	estado.h, structs.cpp y .h
Disco	lectura/escritura de MBR, EBR y búsqueda de particiones.	disk_utils.cpp y .h
EXT2	acceso al superbloque, bitmaps, inodos, bloques, rutas, usuarios y permisos	ext2_utils.cpp y .h
Comandos	comandos a ejecutar sobre el disco	mkdisk, rmdisk, fdisk, mount, mounted, mkfs, login, logout, mkgro, etc.
Reportes	REP: mbr, disk, inode, block, bitmaps, tree, sb, file y ls	reports.cpp y .h
HTTP / API	exposición del backend mediante HTTP / JSON	api_server_main.cpp, http_server.cpp y .h, json:util.cpp y .h

Comandos utilizados:

Los ejemplos son orientativos; la obligatoriedad exacta de parámetros y las opciones aceptadas dependen del analizador y del validador del proyecto

Comando	Función / efecto	Ejemplo
MKDISK	Crea el archivo .mia y su MBR.	MKDISK -size=20 -unit=M -path="/discos/Disco1.mia"
RMDISK	Elimina un disco simulado tras la confirmación requerida.	RMDISK -path="/discos/Disco1.mia"
FDISK	Crea, elimina o ajusta particiones; modifica MBR/EBR.	FDISK -size=5 -unit=M -path="/discos/Disco1.mia" -name=datos
MOUNT	Monta una partición y genera un ID temporal.	MOUNT -path="/discos/Disco1.mia" -name=datos
UNMOUNT	Elimina de RAM el montaje asociado al ID.	UNMOUNT -id=091A
MKFS	Formatea e inicializa las estructuras; fs selecciona EXT2/EXT3.	MKFS -id=091A -fs=3fs
LOGIN	Valida credenciales y crea sesión en RAM.	LOGIN -user=root -pass=123 -id=091A
LOGOUT	Cierra la sesión activa.	LOGOUT
MKGRP	Crea un grupo.	MKGRP -name=desarrollo
RMGRP	Elimina o marca eliminado un grupo según la implementación.	RMGRP -name=desarrollo
MKUSR	Registra un usuario asociado a un grupo.	MKUSR -user=ana -pass=abc123 -grp=desarrollo
RMUSR	Elimina o marca eliminado un usuario.	RMUSR -user=ana
CHGRP	Cambia el grupo de un usuario.	CHGRP -user=ana -grp=desarrollo
MKDIR	Crea un directorio; -p puede crear padres intermedios.	MKDIR -path=/home/ana/docs -p
MKFILE	Crea un archivo y asigna inodo/bloques.	MKFILE -path=/home/ana/docs/a.txt -size=120
CAT	Lee el contenido de un archivo, sujeto a permisos.	CAT -file=/home/ana/docs/a.txt
RENAME	Cambia el nombre de un archivo/directorio.	RENAME -path=/home/ana/a.txt -name=b.txt
COPY	Copia un elemento a una ruta destino admitida.	COPY -path=/home/ana/a.txt -dest=/home/ana/copia.txt
MOVE	Mueve un elemento y actualiza directorios involucrados.	MOVE -path=/home/ana/a.txt -dest=/home/ana/docs/a.txt
REMOVE	Elimina un elemento y libera recursos según implementación.	REMOVE -path=/home/ana/docs/a.txt
FIND	Busca nombres que coincidan con un patrón.	FIND -path=/home/ana -name="*.txt"
CHOWN	Cambia el propietario de un archivo/directorio.	CHOWN -path=/home/ana/docs/a.txt -user=root
CHMOD	Modifica permisos con los parámetros aceptados.	CHMOD -path=/home/ana/docs/a.txt -ugo=640
LOSS	Simula pérdida de estructuras; usar solo en discos de prueba.	LOSS -id=091A
REP	Genera reportes del disco y del sistema de archivos.	REP -name=mbr -path=/reportes/mbr.dot -id=091A

- MKDISK / FDISK: modifican el disco, MBR y, cuando corresponde, EBR.
- MKFS: inicializa superbloque, bitmaps, inodos y bloques; EXT3 prepara además el journaling definido por el proyecto.
- MKDIR / MKFILE: asignan inodos/bloques y actualizan bitmaps y directorios.
- CAT / FIND / REP: consultan contenido y estructuras.
- REMOVE / RENAME / COPY / MOVE: actualizan entradas de directorio, asignaciones y/o contenido; las operaciones mutadoras admitidas se registran en journaling.
- LOGIN / LOGOUT / usuarios: administran sesión en RAM y registros de usuarios/grupos persistidos.
- UNMOUNT: elimina el montaje en memoria, pero no borra el .mia.

estado.h (permanece en RAM)

- discos: cache del MBR de discos conocidos; la fuente persistente real es el .mia.
- discosMontados: MOUNT es deliberadamente RAM-only.
- particionesFormateadas: cache del estado de formato.
- Las estructuras antiguas usuarios, archivos y carpetas permanecen por compatibilidad con la fase anterior; ya no son la
- fuente principal del contenido persistente.
- sesion: usuario, id, uid y gid de la sesión actual; permanece en RAM

structs.h (estructuras binarias)

Las estructuras que se escriben directamente en .mia no contienen std::string ni std::vector. Esto es importante porque el archivo binario necesita una representación de tamaño fijo y directamente serializable.

Analizador (léxico y sintáctico)

El analizador utiliza validarLexico() para verificar que comando y nombres de parámetros tengan una forma léxica válida, validarSintaxis() y validarComillasCerradas() sirve para verificar que los parámetros tengan una estructura válida y las comillas están cerradas.

Tokenización

lexical_analysis() recorre la línea carácter por carácter. Identifica el comando, parámetros que comienzan con '-', valores y cadenas entre comillas. Por ejemplo, una ruta como -path="/tmp/Mi Disco.mia" puede conservar el espacio interno.

Despacho

despacharComando() convierte el nombre del comando a minúsculas y selecciona la función correspondiente: mkdisk(), fdisk(), mount(), mkfs(), login(), mkfile(), mkdir(), cat(), rep(), etc.

Archivo .smia

procesar_archivo_comandos() separa el contenido en líneas, conserva el número original de línea, ignora líneas vacías y comentarios que empiezan con # y procesa cada línea independientemente. Un error en una línea no obliga a detener el procesamiento de las siguientes.

Simulación EXT2 / EXT3: cómo se representa en el archivo binario .mia

Esta es la diferencia fundamental respecto de la primera versión del proyecto. El archivo .mia es ahora el almacenamiento persistente de la simulación. El programa usa operaciones binarias seekg/seekp/read/write para acceder a posiciones concretas del disco simulado.

MKDISK: creación física del disco

MKDISK valida tamaño, unidad, fit, ruta y extensión .mia. Luego crea las carpetas padre si son necesarias, crea el archivo, lo llena con ceros hasta alcanzar el tamaño solicitado y escribe un MBR al comienzo

- MBR: El MBR se escribe al offset 0 del .mia. Contiene tamaño total, fecha, firma, fit y las cuatro entradas de partición. disk_utils.cpp ofrece leerMBR() y escribirMBR().
- FDISK: primero lee el MBR del .mia, calcula espacios disponibles y aplica first fit, best fit o worst fit según el fit correspondiente. Cuando encuentra una ubicación, actualiza la entrada de partición y vuelve a escribir el MBR. En una extendida también puede escribir EBRs físicamente dentro de la región extendida.
- MOUNT: relaciona una partición existente del .mia con un ID de montaje almacenado en RAM. En esta versión el montaje no se persiste dentro del archivo; al reiniciar el programa se debe volver a montar. El ID utiliza la convención configurada para el proyecto, con base de mi carnet (09).

Estructuras de datos

El archivo .mia contiene el disco simulado como una secuencia de bytes. Los offsets y tamaños permiten localizar las estructuras
dentro del archivo.



MBR (Master Boot Record)

Está al inicio del .mia (offset 0). Contiene metadatos del disco y entradas de particiones, según la estructura implementada. MKDISK crea el disco y escribe el MBR; FDISK actualiza la información de particionado.

EBR (Extended Boot Record)

Describe particiones lógicas dentro de una partición extendida. Se ubica en esa región y puede enlazar otros EBR para recorrer particiones lógicas. Los campos exactos son los definidos por las estructuras C++ del proyecto.

Superbloque

MKFS escribe el superbloque al formatear. Guarda metadatos globales, contadores y referencias/offsets que permiten localizar bitmaps, tabla de inodos y bloques.

Bitmaps

Los bitmaps indican qué inodos y bloques están ocupados o libres. Crear archivos/directorios asigna recursos y actualiza los bits y contadores; eliminar contenido libera los recursos correspondientes.

Inodos

Un inodo describe un archivo o directorio. La implementación documentada incluye UID, GID, tamaño, marcas de tiempo, referencias a bloques, tipo y permisos. Las referencias directas e indirectas permiten localizar el contenido.

Bloques

Los bloques guardan contenido de archivos o entradas de directorios. Un bloque de carpeta relaciona nombres con números de inodo; un bloque de archivo almacena bytes. Los archivos grandes pueden requerir referencias indirectas.

Journaling de EXT3

EXT3 agrega un registro de operaciones que modifican el sistema de archivos. El proyecto contempla una capacidad fija de 50 entradas. El registro permite inspeccionar operaciones mutadoras admitidas; no implica recuperación transaccional completa salvo que esta esté implementada explícitamente

Cálculo utilizado por MKFS

sb = sizeof(sbloque)
ino = sizeof(inodo)
bc = sizeof(bcarpetas)
n = (size - sb) / (1 + 3 + ino + 3*bc)
blocks = 3*n

El código actual calcula el número de inodos con esa expresión y utiliza tres veces esa cantidad como número de bloques. Luego determina offsets consecutivos para bitmap de inodos, bitmap de bloques, tabla de inodos y bloques.

Campos de ubicación del superbloque

s_bm_inode_start = start + sizeof(sbloque)
s_bm_block_start = s_bm_inode_start + n
s_inode_start = s_bm_block_start + blocks
s_block_start = s_inode_start + n*sizeof(inodo)

Estos offsets permiten que las funciones de EXT2 encuentren una estructura con una operación del tipo: offset = s_block_start + índice * tamaño_del_bloque.

Inodos, bloques y bitmaps

El sistema utiliza estructuras binarias de tamaño fijo. Los índices de los bitmaps identifican inodos y bloques físicos dentro
de la partición EXT2.

Inodo

El inodo contiene UID, GID, tamaño, tiempos de acceso/creación/modificación, 15 referencias de bloque, tipo y permisos.
i_type=0 representa carpeta e i_type=1 archivo.

Bloque de carpeta

bcarpetas tiene cuatro entradas content. Cada entrada almacena nombre y número de inodo. MKFS crea la raíz con '.' y '..' y agrega users.txt como entrada inicial.

Bloque de archivo

barchivos contiene 64 bytes de contenido. Cuando un archivo necesita más bloques, ext2_utils calcula el índice lógico y utiliza referencias directas, indirectas simples y dobles; la lectura contempla también la estructura de triple indirección.

Asignación

- asignarInodo() busca el primer byte 0 del bitmap de inodos, lo cambia a 1 y actualiza contadores del superbloque.
- asignarBloque() hace lo mismo con el bitmap de bloques.
- liberarInodo() y liberarBloque() vuelven a marcar el elemento como libre y actualizan el superbloque.

Usuarios, grupos, archivos y permisos

La información persistente de usuarios y grupos está dentro de /users.txt en la partición EXT2. La sesión actual sí se mantiene en RAM.

MKFS crea users.txt

1, G, root
1, U, root, root, 123

MKFS crea el inodo de la raíz, el inodo de users.txt, sus bloques y las entradas correspondientes en la carpeta raíz.

LOGIN

LOGIN busca el ID montado, abre la partición EXT2, localiza /users.txt, lee su contenido y verifica usuario, grupo y contraseña. Si es correcto, guarda en estado.sesion el usuario, ID, UID y GID.

MKGRP / RMGRP / MKUSR / RMUSR / CHGRP

Estos comandos requieren sesión activa y actualizan físicamente /users.txt. Las operaciones de eliminación marcan los
registros como eliminados mediante el mecanismo implementado en el archivo de usuarios.

MKDIR y MKFILE

- MKDIR asigna un inodo y un bloque de carpeta, crea '.' y '..' y agrega la entrada en el directorio padre.
- MKFILE asigna un inodo y tantos bloques de archivo como sean necesarios. -cont tiene prioridad sobre -size; con -size se
- genera la secuencia 0123456789... .
- CAT lee inodos/bloques del .mia y valida permisos de lectura

Permisos

ext2_utils.cpp interpreta permisos UGO a partir de i_perm y considera al usuario root con UID 1 como privilegiado. El permiso efectivo se selecciona según si el UID coincide con el propietario, si el GID coincide con el grupo o, en otro caso, usando los bits de otros.

Reportes REP

reports.cpp lee el .mia y genera los reportes solicitados. Los reportes gráficos se construyen como DOT y se renderizan con Graphviz; los bitmaps y file producen texto directamente.

Reporte	Fuente principal	Contenido
mbr	MBR + EBR	Información del disco, MBR y EBRs.
disk	MBR	Distribución del disco y porcentajes.
inode	Bitmap + inodos	Solo inodos utilizados.
block	Bitmap + inodos + bloques	Solo bloques utilizados.
bm_inode	Bitmap de inodos	Todos los bits, 20 por línea.
bm_block / bm_bloc	Bitmap de bloques	Todos los bits, 20 por línea.
tree	Inodos + bloques	Árbol EXT2 completo alcanzable.
sb	Superbloque	Campos del superbloque.
file	Inodo + bloques	Contenido de un archivo.
ls	Inodo + directorios	Nombre, tipo, permisos, UID, GID, tamaño y fechas.

Interfaz gráfica

Se utiliza React + Vite. La interfaz actual está construida con React y Vite. React administra el estado de la pantalla y Vite proporciona el entorno de desarrollo/build. El Frontend no contiene la lógica de EXT2: su función principal es interacción con el usuario y comunicación con la API.

App.jsx

- comandos: contiene el texto que el usuario escribió o el contenido leído desde un archivo .smia.
- resultados: contiene la respuesta textual devuelta por el backend.
- procesando: evita ejecutar otra petición mientras una está en curso y cambia el botón a “Procesando...” .
- archivo Seleccionado: guarda el nombre del .smia que se cargó.
- El botón Subir .smia usa FileReader para leer el archivo como texto. No trata un .smia como disco.
- El botón Limpiar borra comandos, resultados y nombre del archivo.
- El botón Procesar llama a procesarArchivo(comandos).

api.js (puente entre React y C++)

Este archivo concentra la comunicación de red. La aplicación usa fetch nativo del navegador; no necesita una biblioteca
HTTP adicional.



```
1  const URL_BASE = 'http://localhost:8080'
2
3  export async function procesarArchivo(contenido) {
4    const respuesta = await fetch(`${URL_BASE}/api/procesar`, {
5      method: 'POST',
6      headers: { 'Content-Type': 'application/json' },
7      body: JSON.stringify({ contenido }),
8    })
9
10   const datos = await respuesta.json()
11
12   if (!respuesta.ok) {
13     throw new Error(datos.error || 'Error desconocido del backend')
14   }
15
16   return datos.reporte
17 }
18
```

main.jsx, index.htm, App.cpp y package.json

- main.jsx monta el componente App dentro de #root y carga App.css.
- index.html es el documento base que Vite sirve al navegador.
- App.css contiene la presentación visual de la interfaz.
- package.json declara React, ReactDOM y Vite; los scripts principales son dev, build y preview.

Archivo .smia

El archivo .smia es un archivo de comandos. React lo lee con FileReader y copia su contenido al textarea. Después, el usuario presiona Procesar y ese texto se manda a /api/procesar. El .smia no se envía como un disco binario ni se convierte en .mia.

API REST y comunicación Frontend/Backend

El backend ejecutable api_server levanta un servidor HTTP propio en el puerto 8080. La implementación está hecha en C++ usando sockets POSIX; no depende de un framework HTTP externo. api_server_main.cpp registra las rutas y conecta cada ruta con una función manejadora

POST / api / procesar paso a paso

El frontend se queda escuchando el puerto 8080, se manda por medio de un JSON el comando o contenido hacia el backend, ahi el backend extrae cada campo del JSON, procesa el archivo, lo ejecuta y manda la salida por un JSON al frontend nuevamente

http_server.cpp

- Abre un socket y escucha en 8080.
- Recibe la petición HTTP y separa método, ruta y cuerpo.
- Busca la ruta registrada y ejecuta el callback correspondiente.
- Construye la respuesta HTTP y la devuelve al navegador.
- ncluye cabeceras CORS y atiende OPTIONS para permitir la comunicación entre el puerto del Frontend y el 8080.

json_util.cpp

El proyecto no utiliza una biblioteca JSON externa para estas peticiones simples. extraer_campo_json() obtiene campos de entrada y escapar_json() prepara texto para incorporarlo correctamente al JSON de salida.

La comunicación, por tanto, no es “React llama directamente a mkdisk.cpp”. La cadena correcta es: **React → fetch → HTTP server → API handler → analizador → comando → archivo .mia/RAM → respuesta → React**.

MAKEFILE

El archivo se llama Makefile. Es un archivo de reglas para automatizar la compilación. En lugar de escribir manualmente un comando g++ enorme cada vez, se ejecuta make y el sistema utiliza las reglas declaradas

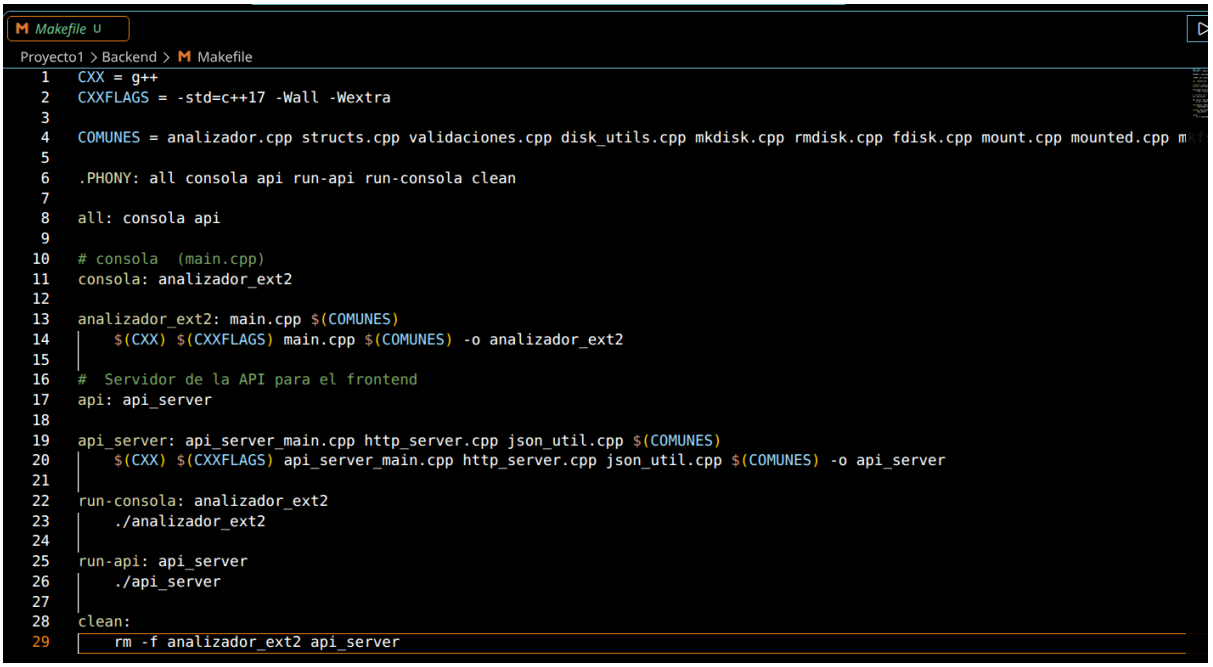
Variables y módulos comunes

CXX = g++
CXXFLAGS = -std=c++17 -Wall -Wextra
COMUNES = analizador.cpp structs.cpp validaciones.cpp disk_utils.cpp …

COMUNES contiene los módulos que comparten los dos ejecutables: el modo consola y el servidor de la API. Esto evita mantener dos copias de la lógica.
comandos del Makefile

Comando	Responsabilidad
make	compila la api_server
make run-consola	compilar el ejecutable de consola y lo ejecuta
make run-api	compila api_server y deja el servidor escuchando 8080
make clean	elimina los binarios compilados

Si se modifica analizador.cpp, ext2_utils.cpp, mkfile.cpp, reports.cpp o cualquier otro archivo incluido en COMUNES, la regla recompila esos archivos junto con el punto de entrada correspondiente. Para el proyecto es especialmente útil porque consola y API deben utilizar exactamente la misma lógica de comandos.



Plan de recomendaciones

Para el sistema de archivos EXT3 con arquitectura AWS (Frontend en S3 y Backend en EC2)

No	Recomendación	Descripción	Prioridad
1	Proteger el acceso al sistema	Implementar autenticación segura y verificar los permisos de cada usuario antes de ejecutar operaciones sobre archivos y directorios.	Alta
2	Configurar correctamente AWS	Mantener el frontend en Amazon S3 y el backend en una instancia EC2, restringiendo el acceso a los recursos según las necesidades del proyecto.	Alta
3	Asegurar la comunicación	Utilizar HTTPS para las comunicaciones entre el navegador y los servicios publicados, evitando transmitir credenciales sin cifrado.	Alta
4	Validar los comandos	Comprobar los parámetros de cada comando antes de ejecutarlo para prevenir errores y operaciones inválidas.	Alta

5	Implementar copias de seguridad	Crear respaldos periódicos de los archivos de disco virtual `.mia` y de los datos importantes del sistema.	Alta
6	Mejorar el manejo de errores	Mostrar mensajes claros cuando una operación falle, indicando la causa cuando sea posible y evitando cierres inesperados.	Media
7	Monitorear los recursos	Supervisar el uso de CPU, memoria, almacenamiento y tráfico de red de la instancia EC2.	Media
8	Optimizar el frontend	Reducir la carga innecesaria de recursos y mejorar la presentación de mensajes, resultados y errores en la interfaz web.	Media
9	Documentar las operaciones	Mantener actualizada la documentación de los comandos, sus parámetros, sus efectos y los permisos necesarios.	Media
10	Realizar pruebas periódicas	Verificar el funcionamiento de los comandos, la persistencia de los datos y la comunicación entre el frontend y el backend.	Alta

Acciones recomendadas para su implementación

- ➔ **A corto plazo:** configurar las reglas de acceso de AWS, validar las entradas de los comandos, proteger las credenciales y establecer copias de seguridad.
- ➔ **A mediano plazo:** incorporar monitoreo, mejorar los mensajes de error y realizar pruebas de carga y concurrencia.
- ➔ **A largo plazo:** automatizar las copias de seguridad, establecer procedimientos de recuperación y evaluar las posibilidades de escalabilidad de la arquitectura.

Plan de mantenimiento

Objetivo: Establecer actividades preventivas y correctivas que permitan conservar el funcionamiento del sistema de archivos EXT3 y de su infraestructura en AWS.

Mantenimiento preventivo

Consiste en realizar revisiones programadas para evitar fallos y reducir el riesgo de pérdida de información.

Actividad	Procedimiento	Frecuencia recomendada	Responsable
Verificación del backend	Comprobar que la API en C++ responda correctamente y ejecute los comandos esperados.	Semanal	Administrador del sistema
Revisión del frontend	Verificar que la aplicación alojada en S3 cargue correctamente y pueda comunicarse con el backend.	Semanal	Desarrollador frontend
Revisión de registros	Examinar los registros de ejecución para identificar errores, solicitudes fallidas o comportamientos anormales.	Semanal	Administrador del sistema
Copias de seguridad	Respaldar los archivos `.mia` y los datos necesarios para recuperar el sistema.	Diaria o según la frecuencia de cambios	Administrador del sistema
Revisión de almacenamiento	Comprobar el espacio disponible en el almacenamiento de EC2 y detectar archivos innecesarios.	Mensual	Administrador del sistema
Actualización de dependencias	Revisar las versiones de las bibliotecas, herramientas y componentes utilizados por el proyecto.	Mensual	Equipo de desarrollo

Revisión de seguridad	Verificar las reglas de acceso, los permisos de los usuarios y la protección de las credenciales.	Mensual	Administrador del sistema
Pruebas de recuperación	Restaurar una copia de seguridad en un entorno de pruebas y comprobar que los datos sean utilizables.	Trimestral	Equipo de desarrollo

Las frecuencias son propuestas iniciales y deben ajustarse al uso real del sistema y a la importancia de los datos.

Mantenimiento correctivo

Se realiza cuando ocurre un error o una falla que afecta el funcionamiento del sistema.

Problema detectado	Acción correctiva
El frontend no carga	Verificar el contenido publicado en S3, la configuración del alojamiento y los permisos de acceso.
El frontend no se comunica con el backend	Comprobar la URL de la API, la conectividad de EC2, las reglas del firewall o grupo de seguridad y la configuración CORS.
La API deja de responder	Revisar los registros del proceso, identificar la causa del fallo y reiniciar el servicio si corresponde.
Un comando devuelve resultados incorrectos	Revisar la validación de parámetros, la lógica del comando y las estructuras del sistema de archivos involucradas.
Los cambios no se conservan	Verificar los procedimientos de escritura, sincronización y persistencia del archivo `.mia`.
Un usuario no puede realizar una operación	Revisar su autenticación, sus permisos y las reglas de autorización aplicadas al comando.

El archivo `.mia` presenta daños	Detener las operaciones de escritura, conservar una copia del archivo afectado y evaluar la restauración desde un respaldo verificado.
La instancia EC2 presenta problemas de rendimiento	Examinar el consumo de CPU, memoria, almacenamiento y red para determinar si se requiere optimización o ampliación de recursos.

Importante: antes de realizar reparaciones que modifiquen el archivo `.mia`, se debe conservar una copia del estado actual. La restauración de un respaldo puede descartar operaciones recientes, por lo que es necesario verificar qué información se recuperará.

Mantenimiento evolutivo

Este mantenimiento incorpora mejoras cuando cambian los requisitos del proyecto.

Entre las actividades propuestas se encuentran:

- Incorporar nuevos comandos para la administración de archivos y directorios.
- Mejorar la administración de usuarios, grupos y permisos.
- Optimizar las estructuras de datos y los algoritmos de búsqueda.
- Ampliar las pruebas automatizadas del backend y del frontend.
- Mejorar los reportes sobre el estado del sistema de archivos.
- Adaptar la infraestructura de AWS a futuras necesidades de capacidad y seguridad.

Procedimiento de mantenimiento

Para realizar cualquier intervención, se recomienda seguir este orden:

1. **Identificar el problema:** registrar el error, la operación afectada y las condiciones en las que ocurre.
2. **Evaluar el impacto:** determinar si afecta al frontend, al backend, al almacenamiento o a los permisos.
3. **Proteger la información:** crear una copia de seguridad antes de modificar datos o estructuras importantes.
4. **Aplicar la solución:** corregir el componente responsable del problema.
5. **Ejecutar pruebas:** comprobar que los comandos y las operaciones afectadas funcionen correctamente.
6. **Verificar la persistencia:** confirmar que los cambios se conserven después de reiniciar el servicio o volver a abrir el sistema.
7. **Documentar la intervención:** registrar la causa, la solución aplicada y los resultados obtenidos.

Indicadores para evaluar el mantenimiento

Para determinar si los planes están dando resultados, se pueden utilizar los siguientes indicadores:

- **Disponibilidad del sistema:** porcentaje del tiempo durante el cual la aplicación está operativa.
- **Tasa de errores:** proporción de operaciones que terminan con errores respecto del total de operaciones ejecutadas.
- **Éxito de las copias de seguridad:** porcentaje de respaldos completados correctamente.
- **Tiempo de recuperación:** tiempo necesario para restablecer el servicio después de una falla.
- **Integridad de los datos:** capacidad de conservar y recuperar correctamente la información almacenada.
- **Cumplimiento del mantenimiento:** porcentaje de actividades preventivas realizadas dentro del período programado.